

```

import numpy as np
import matplotlib.pyplot as plt
import warnings
warnings.filterwarnings('ignore')

from sympy import Symbol
from sympy import init_printing
from sympy import integrate
from sympy import limit
from sympy import diff

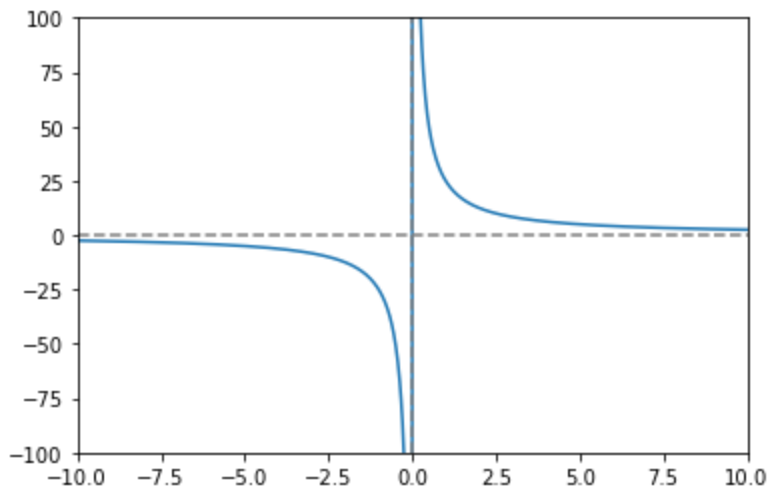
# init_printing(use_unicode=False, wrap_line=False)

# x = Symbol('x')

def inf_function(x):
    return 25/x
x = np.linspace(-10,10,1000)
y = inf_function(x)
plt.plot(x,y)
plt.axvline(x = 0, color = "grey", linestyle= "--")
plt.axhline(y = 0, color = "grey", linestyle= "--")
plt.xlim(-10,10)
plt.ylim(-100,100)

plt.show()

```



```

x = Symbol("x")

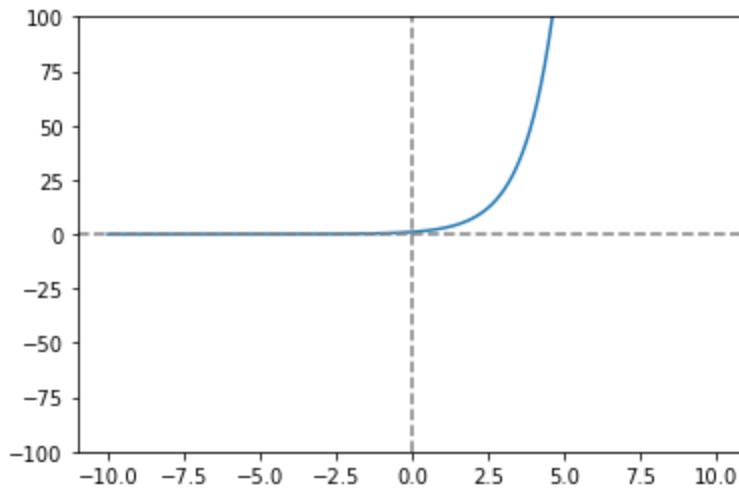
limit(1/x, x,0)

```

```
plt.axhline(y = 0, color = "grey", linestyle= "--")
```

```
plt.ylim(-20,20)
plt.ylim(-100,100)
```

```
plt.show()
```



```
def log_function(x):
```

```
    return np.log(x)
```

```
x1 = np.linspace(-100,100,10000)
```

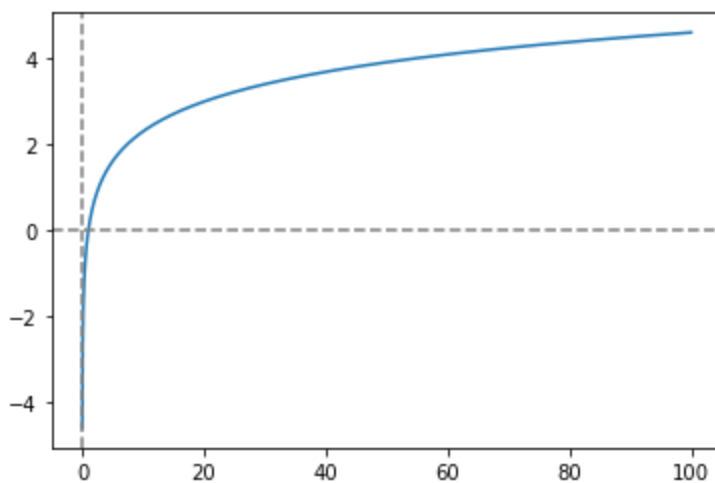
```
y = log_function(x1)
```

```
plt.plot(x1,y)
```

```
plt.axvline(x = 0, color = "grey", linestyle= "--")
```

```
plt.axhline(y = 0, color = "grey", linestyle= "--")
```

```
plt.show()
```



```
def sigmoid_function(x):

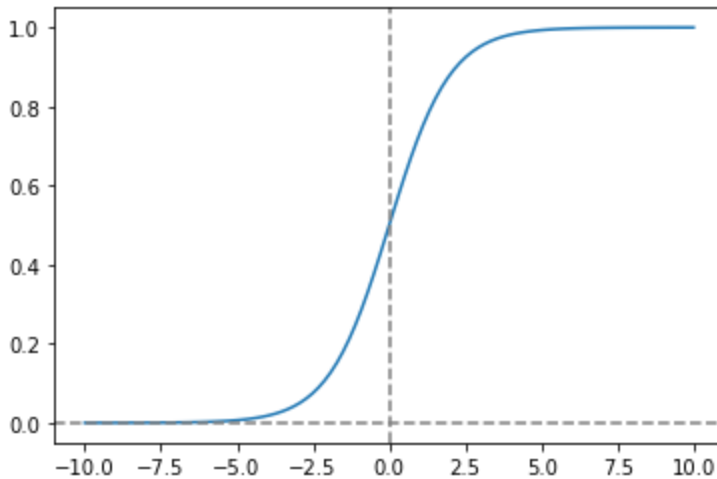
    return 1 / (1+(np.e)**(-x) )

x1 = np.linspace(-10,10,100)

y = sigmoid_function(x1)
plt.plot(x1,y)

plt.axvline(x = 0, color = "grey", linestyle= "--")
plt.axhline(y = 0, color = "grey", linestyle= "--")

plt.show()
```



differentiation :

what is the rate of change of y when we change x :

$$m = \frac{\text{change in } y}{\text{change in } x} = \frac{\Delta y}{\Delta x} = \frac{y_2 - y_1}{x_2 - x_1}$$

```
def f(x):
    return x**2
x = np.linspace(-10,10,1000)
y = f(x)

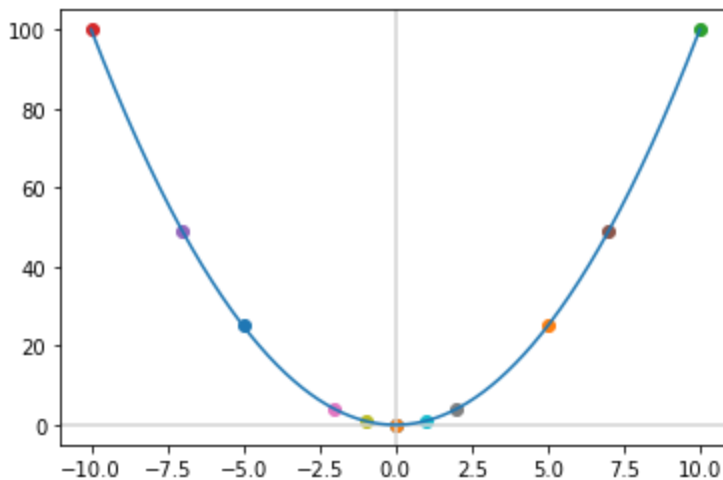
def f(x):
    return x**2
x = np.linspace(-10,10,1000)
y = f(x)

q = -5
p = 5
fig, ax = plt.subplots()
plt.axvline(x=0, color='lightgray')
```

```
plt.axhline(y=0, color='lightgray')
plt.scatter(q,f(q))
plt.scatter(p,f(p))
plt.scatter(10,f(10))
plt.scatter(-10,f(-10))
plt.scatter(-7,f(-7))
plt.scatter(7,f(7))
plt.scatter(-2,f(-2))
plt.scatter(2,f(2))
plt.scatter(-1,f(-1))
plt.scatter(1,f(1))
plt.scatter(0,f(0))
plt.scatter(0,f(0))

ax.plot(x,y)

[<matplotlib.lines.Line2D at 0x16f7144bd00>]
```



As Q becomes infinitely close to P :

- $x_2 - x_1$ approaches 0
- In other words, Δx approaches 0
- This can be denoted as $\Delta x \rightarrow 0$

Using the delta method, we've derived the definition of differentiation from first principles. The derivative of y (denoted dy) with respect to x (denoted dx) can be represented as:

$$\frac{dy}{dx} = \lim_{\Delta x \rightarrow 0} \frac{\Delta y}{\Delta x}$$

Expanding Δy out to $y_2 - y_1$:

$$\frac{dy}{dx} = \lim_{\Delta x \rightarrow 0} \frac{y_2 - y_1}{\Delta x}$$

Finally, replacing y_1 with $f(x)$ and replacing y_2 with $f(x + \Delta x)$, we obtain a common representation of differentiation:

$$\frac{dy}{dx} = \lim_{\Delta x \rightarrow 0} \frac{f(x + \Delta x) - f(x)}{\Delta x}$$

Derivative :

In mathematics, the derivative of a function of a real variable measures the sensitivity to change of the function value (output value) with respect to a change in its argument (input value).

Definition

A function of a real variable $f(x)$ is differentiable at a point a of its domain, if its domain contains an open interval I containing a , and the limit

$$L = \lim_{h \rightarrow 0} \frac{f(a+h) - f(a)}{h}$$

exists. This means that, for every positive real number ε (even very small), there exists a positive real number δ such that, for every h such that $|h| < \delta$ and $h \neq 0$ then $f(a+h)$ is defined, and

$$\left| L - \frac{f(a+h) - f(a)}{h} \right| < \varepsilon,$$

where the vertical bars denote the absolute value (see (ε , δ)-definition of limit).

If the function f is differentiable at a , that is if the limit L exists, then this limit is called the *derivative* of f at a , and denoted $f'(a)$ (read as " f prime of a ") or $\frac{df}{dx}(a)$ (read as "the derivative of f with respect to x at a ", " dy by dx at a ", or " dy over dx at a "); see § Notation (details), below.

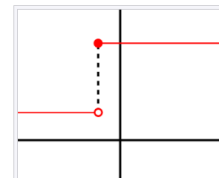
Continuity and differentiability

If f is differentiable at a , then f must also be continuous at a . As an example, choose a point a and let f be the step function that returns the value 1 for all x less than a , and returns a different value 10 for all x greater than or equal to a . f cannot have a derivative at a . If h is negative, then $a+h$ is on the low part of the step, so the secant line from a to $a+h$ is very steep, and as h tends to zero the slope tends to infinity. If h is positive, then $a+h$ is on the high part of the step, so the secant line from a to $a+h$ has slope zero. Consequently, the secant lines do not approach any single slope, so the limit of the difference quotient does not exist.

However, even if a function is continuous at a point, it may not be differentiable there. For example, the absolute value function given by $f(x) = |x|$ is continuous at $x = 0$, but it is not differentiable there. If h is positive, then the slope of the secant line from 0 to h is one, whereas if h is negative, then the slope of the secant line from 0 to h is negative one. This can be seen graphically as a "kink" or a "cusp" in the graph at $x = 0$. Even a function with a smooth graph is not differentiable at a point where its tangent is vertical: For instance, the function given by $f(x) = x^{1/3}$ is not differentiable at $x = 0$.

In summary, a function that has a derivative is continuous, but there are continuous functions that do not have a derivative.

Most functions that occur in practice have derivatives at all points or at almost every point. Early in the history of calculus, many mathematicians assumed that a continuous function was differentiable at most points. Under mild conditions, for example if the function is a monotone function or a Lipschitz function, this is true. However, in 1872 Weierstrass found the first example of a function that is continuous everywhere but differentiable nowhere. This example is now known as the Weierstrass function. In 1931, Stefan Banach proved that the set of functions that have a derivative at some point is a meager set in the space of all continuous functions.^[1] Informally, this means that hardly any random continuous functions have a derivative at even one point.



This function does not have a derivative at the marked point, as the function is not continuous there (specifically, it has a jump discontinuity).



The absolute value function is continuous, but fails to be differentiable at $x = 0$ since the tangent slopes do not approach the same value from the left as they do from the right.

https://upload.wikimedia.org/wikipedia/commons/2/2d/Tangent_function_animation.gif

Elementary rules of differentiation

Constant Term Rule [\[edit \]](#)

For any value of c , where $c \in \mathbb{R}$, for any value of $x \in \mathbb{R}$, $\frac{d}{dx}(c) = 0$.^[4]

Proof [\[edit \]](#)

Let $c \in \mathbb{R}$. Now, we will prove, from first principles, what the derivative is. Let $x \in \mathbb{R}$.

For clarity, we will define the new function f , so that $f(x) = c$. We want to show that

$\frac{d}{dx}(c) = 0$. From what we defined what $f(x)$ was, we can rewrite the statement:

$$\begin{aligned} & \frac{d}{dx}(c) \\ &= \frac{d}{dx}f(x) = f'(x) \end{aligned}$$

This is just done for clarity of notation. This way, we can use the definition of the derivative to find $f'(x)$.

$$\begin{aligned} f'(x) &= \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h} \\ &= \lim_{h \rightarrow 0} \frac{(c) - (c)}{h} \\ &= \lim_{h \rightarrow 0} \frac{0}{h} \\ &= \lim_{h \rightarrow 0} 0 \\ &= 0 \end{aligned}$$

Thus, we have found out that: for any value of $c \in \mathbb{R}$, for any value of $x \in \mathbb{R}$, $\frac{d}{dx}(c) = 0$.

Differentiation is linear [\[edit \]](#)

Main article: [Linearity of differentiation](#)

For any functions f and g and any real numbers a and b , the derivative of the function $h(x) = af(x) + bg(x)$ with respect to x is: $h'(x) = af'(x) + bg'(x)$.

In [Leibniz's notation](#) this is written as:

$$\frac{d(af + bg)}{dx} = a \frac{df}{dx} + b \frac{dg}{dx}.$$

Special cases include:

- The constant factor rule

$$(af)' = af'$$

- The sum rule

$$(f + g)' = f' + g'$$

- The subtraction rule

$$(f - g)' = f' - g'.$$

The chain rule [\[edit \]](#)

Main article: [Chain rule](#)

The derivative of the function $h(x) = f(g(x))$ is

$$h'(x) = f'(g(x)) \cdot g'(x).$$

In Leibniz's notation, this is written as:

$$\frac{d}{dx} h(x) = \frac{d}{dz} f(z) \Big|_{z=g(x)} \cdot \frac{d}{dx} g(x),$$

often abridged to

$$\frac{dh(x)}{dx} = \frac{df(g(x))}{dg(x)} \cdot \frac{dg(x)}{dx}.$$

Focusing on the notion of maps, and the differential being a map D , this is written in a more concise way as:

$$[D(f \circ g)]_x = [Df]_{g(x)} \cdot [Dg]_x.$$

The inverse function rule [\[edit \]](#)

Main article: [Inverse functions and differentiation](#)

If the function f has an [inverse function](#) g , meaning that $g(f(x)) = x$ and $f(g(y)) = y$, then

$$g' = \frac{1}{f' \circ g}.$$

In Leibniz notation, this is written as

$$\frac{dx}{dy} = \frac{1}{\frac{dy}{dx}}.$$

Power laws, polynomials, quotients, and reciprocals [\[edit \]](#)

The polynomial or elementary power rule [\[edit \]](#)

Main article: [Power rule](#)

If $f(x) = x^r$, for any real number $r \neq 0$, then

$$f'(x) = rx^{r-1}.$$

When $r = 1$, this becomes the special case that if $f(x) = x$, then $f'(x) = 1$.

Combining the power rule with the sum and constant multiple rules permits the computation of the derivative of any polynomial.

The reciprocal rule [\[edit \]](#)

Main article: [Reciprocal rule](#)

The derivative of $h(x) = \frac{1}{f(x)}$ for any (nonvanishing) function f is:

$$h'(x) = -\frac{f'(x)}{(f(x))^2} \text{ wherever } f \text{ is non-zero.}$$

In Leibniz's notation, this is written

$$\frac{d(1/f)}{dx} = -\frac{1}{f^2} \frac{df}{dx}.$$

The reciprocal rule can be derived either from the quotient rule, or from the combination of power rule and chain rule.

The quotient rule [[edit](#)]

Main article: [Quotient rule](#)

If f and g are functions, then:

$$\left(\frac{f}{g}\right)' = \frac{f'g - g'f}{g^2} \quad \text{wherever } g \text{ is nonzero.}$$

This can be derived from the product rule and the reciprocal rule.

Generalized power rule [[edit](#)]

Main article: [Power rule](#)

The elementary power rule generalizes considerably. The most general power rule is the **functional power rule**: for any functions f and g ,

$$(f^g)' = (e^{g \ln f})' = f^g \left(f' \frac{g}{f} + g' \ln f \right),$$

wherever both sides are well defined.

Special cases

- If $f(x) = x^a$, then $f'(x) = ax^{a-1}$ when a is any non-zero real number and x is positive.
- The reciprocal rule may be derived as the special case where $g(x) = -1$.

Derivatives of exponential and logarithmic functions [[edit](#)]

$$\frac{d}{dx} (c^{ax}) = ac^{ax} \ln c, \quad c > 0$$

the equation above is true for all c , but the derivative for $c < 0$ yields a complex number.

$$\frac{d}{dx} (e^{ax}) = ae^{ax}$$

$$\frac{d}{dx} (\log_c x) = \frac{1}{x \ln c}, \quad c > 1$$

the equation above is also true for all c , but yields a complex number if $c < 0$

$$\frac{d}{dx} (\ln x) = \frac{1}{x}, \quad x > 0.$$

$$\frac{d}{dx} (\ln |x|) = \frac{1}{x}, \quad x \neq 0.$$

$$\frac{d}{dx} (W(x)) = \frac{1}{x + e^{W(x)}}, \quad x > -\frac{1}{e}. \quad \text{where } W(x) \text{ is the [Lambert W function](#)}$$

$$\frac{d}{dx} (x^x) = x^x (1 + \ln x).$$

$$\frac{d}{dx} \left(f(x)^{g(x)} \right) = g(x) f(x)^{g(x)-1} \frac{df}{dx} + f(x)^{g(x)} \ln(f(x)) \frac{dg}{dx}, \quad \text{if } f(x) > 0, \text{ and if } \frac{df}{dx} \text{ and } \frac{dg}{dx} \text{ exist.}$$

$$\frac{d}{dx} \left(f_1(x)^{f_2(x)^{\dots f_n(x)}} \right) = \left[\sum_{k=1}^n \frac{\partial}{\partial x_k} \left(f_1(x_1)^{f_2(x_2)^{\dots f_n(x_n)}} \right) \right] \Big|_{x_1=x_2=\dots=x_n=x}, \quad \text{if } f_{i < n}(x) > 0 \text{ and } \frac{df_i}{dx} \text{ exists.}$$

Derivatives of trigonometric functions [\[edit \]](#)

Main article: [Differentiation of trigonometric functions](#)

$$\begin{array}{ll}(\sin x)' = \cos x = \frac{e^{ix} + e^{-ix}}{2} & (\arcsin x)' = \frac{1}{\sqrt{1-x^2}} \\(\cos x)' = -\sin x = \frac{e^{-ix} - e^{ix}}{2i} & (\arccos x)' = -\frac{1}{\sqrt{1-x^2}} \\(\tan x)' = \sec^2 x = \frac{1}{\cos^2 x} = 1 + \tan^2 x & (\arctan x)' = \frac{1}{1+x^2} \\(\cot x)' = -\csc^2 x = -\frac{1}{\sin^2 x} = -1 - \cot^2 x & (\operatorname{arccot} x)' = \frac{1}{-1-x^2} \\(\sec x)' = \sec x \tan x & (\operatorname{arcsec} x)' = \frac{1}{|x|\sqrt{x^2-1}} \\(\csc x)' = -\csc x \cot x & (\operatorname{arccsc} x)' = -\frac{1}{|x|\sqrt{x^2-1}}\end{array}$$

The derivatives in the table above is for when the range of the inverse secant is $[0, \pi]$ and when the range of the inverse cosecant is $\left[-\frac{\pi}{2}, \frac{\pi}{2}\right]$.

It is common to additionally define an [inverse tangent function with two arguments](#), $\arctan(y, x)$. Its value lies in the range $[-\pi, \pi]$ and reflects the quadrant of the point (x, y) . For the first and fourth quadrant (i.e. $x > 0$) one has $\arctan(y, x > 0) = \arctan(y/x)$. Its partial derivatives are

$$\frac{\partial \arctan(y, x)}{\partial y} = \frac{x}{x^2 + y^2}, \text{ and } \frac{\partial \arctan(y, x)}{\partial x} = \frac{-y}{x^2 + y^2}.$$

Derivatives of hyperbolic functions [\[edit \]](#)

$$\begin{array}{ll}(\sinh x)' = \cosh x = \frac{e^x + e^{-x}}{2} & (\operatorname{arsinh} x)' = \frac{1}{\sqrt{1+x^2}} \\(\cosh x)' = \sinh x = \frac{e^x - e^{-x}}{2} & (\operatorname{arcosh} x)' = \frac{1}{\sqrt{x^2-1}} \\(\tanh x)' = \operatorname{sech}^2 x = \frac{1}{\cosh^2 x} = 1 - \tanh^2 x & (\operatorname{artanh} x)' = \frac{1}{1-x^2} \\(\coth x)' = -\operatorname{csch}^2 x = -\frac{1}{\sinh^2 x} = 1 - \coth^2 x & (\operatorname{arcoth} x)' = \frac{1}{1-x^2} \\(\operatorname{sech} x)' = -\operatorname{sech} x \tanh x & (\operatorname{arsech} x)' = -\frac{1}{x\sqrt{1-x^2}} \\(\operatorname{csch} x)' = -\operatorname{csch} x \coth x & (\operatorname{arcsch} x)' = -\frac{1}{|x|\sqrt{1+x^2}}\end{array}$$

See [Hyperbolic functions](#) for restrictions on these derivatives.

<https://en.wikipedia.org/wiki/Derivative>

https://en.wikipedia.org/wiki/Trigonometric_functions

```
import sympy as sy
```

```
init_printing(use_unicode=False, wrap_line=False)
x = Symbol('x')
```

```
b = Symbol('b')
y = Symbol("y")
m = Symbol("m")
```

```
((y-(m*x)-(b))**2).diff(b)
```

$2b + 2mx - 2y$

```
-2(-b-mx+y)
-2(y-mx-b)
```

$$-(0.1x + 10)^2 + 5(x - 50) + 785$$

```
((-(0.1*x+10)**2) + (5*(x-50)) + 785).diff(x)
```

$3.0 - 0.02x$

```
# d/dx [ ((x-9)**2) + (5*(x-50)) + 785 ]
# = 2x-13
# 2x-13 = 0
# x = 13/2
```

```
((((x-9)**2) + (5*(x-50)) + 785).diff(x))
```

$2x - 13$

```
((((x-9)**2) + (5*(x-50)) + 785).diff(x)).diff(x)
```

2

```
((x+6)**2 + 20).diff(x)
```

$2x + 12$

$$y = a - 4(x-b)^2$$

$$y' = f'(x) = \frac{d}{dx} [a - 4(x-b)^2]$$

$$\begin{aligned} &= \frac{d}{dx} [a - 4(x^2 - 2xb + b^2)] \\ &= \frac{d}{dx} [a - 4x^2 + 8xb - b^2] \\ &= 0 - 8x + 8b - 0 \\ &= -8x + 8b \end{aligned}$$

$$a = 5$$

$$b = 8$$

```
(a-(4*(x-b)**2)).diff(x)
```

$64 - 8x$

$$-(8*x) + (8*b)$$

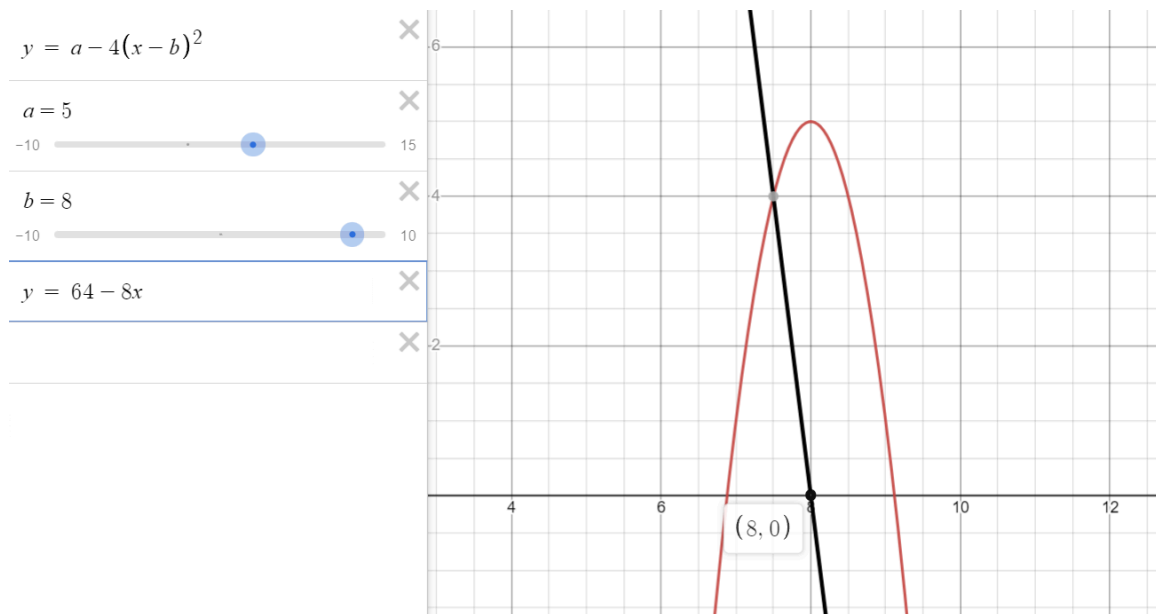
$$64 - 8x$$

$$64 - 8x = 0$$

$$64 = 8x$$

$$x = 8$$

when $x = b$, y will be maximum



```
(3*x**2).diff(x)
```

$$6x$$

```
(x**4).diff(x)
```

$$4x^3$$

```
def f(x):
    return x**2
```

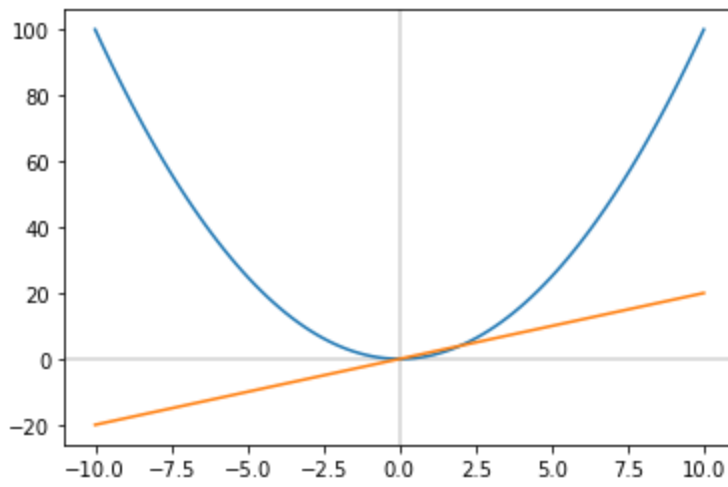
```
x = np.linspace(-10,10,100)
```

```
y = f(x)
```

```
diff_of_y = 2*x
```

```
fig, ax = plt.subplots()
plt.axvline(x=0, color='lightgray')
plt.axhline(y=0, color='lightgray')
ax.plot(x,y)
ax.plot(x,diff_of_y)
```

```
[<matplotlib.lines.Line2D at 0x16f715d7400>]
```



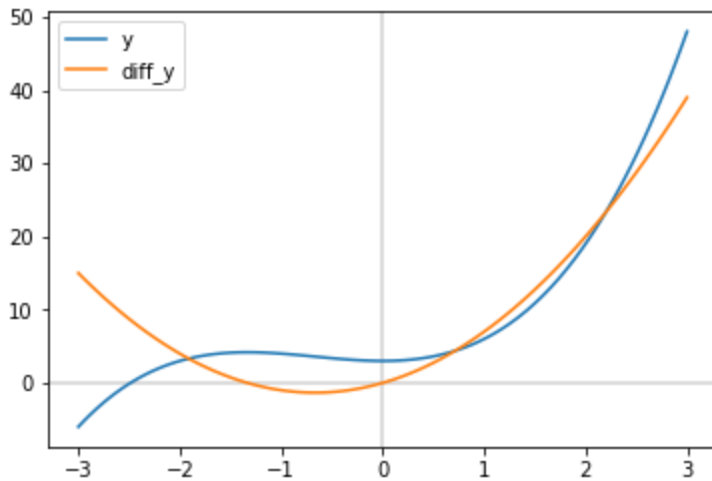
```
def f(x):
    return x**3 + 2*(x**2) + 3
```

```
x = np.linspace(-3,3,100)
```

```
y = f(x)
```

```
diff_of_y = 3*(x**2) + (4*x)
```

```
fig, ax = plt.subplots()
plt.axvline(x=0, color='lightgray')
plt.axhline(y=0, color='lightgray')
ax.plot(x,y,label = "y")
ax.plot(x,diff_of_y,label = "diff_y")
plt.legend()
plt.show()
```



```
] : (-(x+6)**2 + 20).diff(x)
```

```
] :  $-2x - 12$ 
```

```
def f(x):  
    return -(x+6)**2 + 20
```

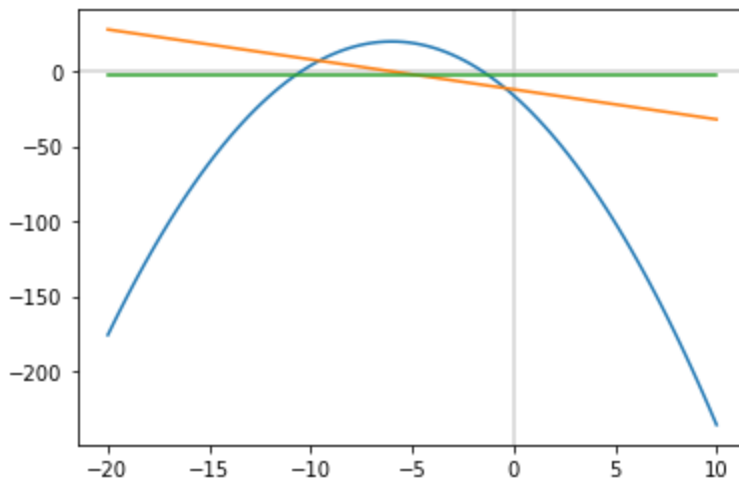
```
x = np.linspace(-20,10,100)
```

```
y = f(x)
```

```
diff_of_y = -2*x -12  
dff_dff_y = -2 * np.ones(shape=x.size)
```

```
fig, ax = plt.subplots()  
plt.axvline(x=0, color='lightgray')  
plt.axhline(y=0, color='lightgray')  
ax.plot(x,y)  
ax.plot(x,diff_of_y)  
ax.plot(x,dff_dff_y)
```

```
[<matplotlib.lines.Line2D at 0x16f716a2520>]
```



```
L] : ((x+6)**2 + 20).diff(x)
```

```
L] :  $2x + 12$ 
```

```
def f(x):
    return (x+6)**2 + 20

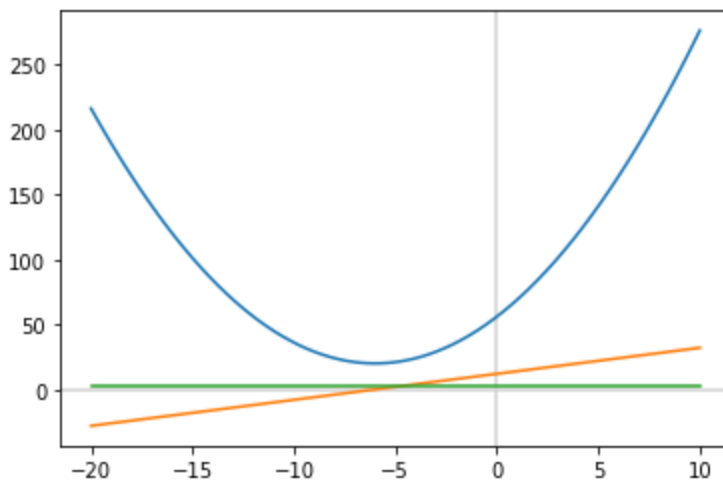
x = np.linspace(-20,10,100)

y = f(x)

diff_of_y = 2*x + 12
dff_dff_y = 2 * np.ones(shape=x.size)

fig, ax = plt.subplots()
plt.axvline(x=0, color='lightgray')
plt.axhline(y=0, color='lightgray')
ax.plot(x,y)
ax.plot(x,diff_of_y)
ax.plot(x,dff_dff_y)
```

[<matplotlib.lines.Line2D at 0x16f7170dc70>]

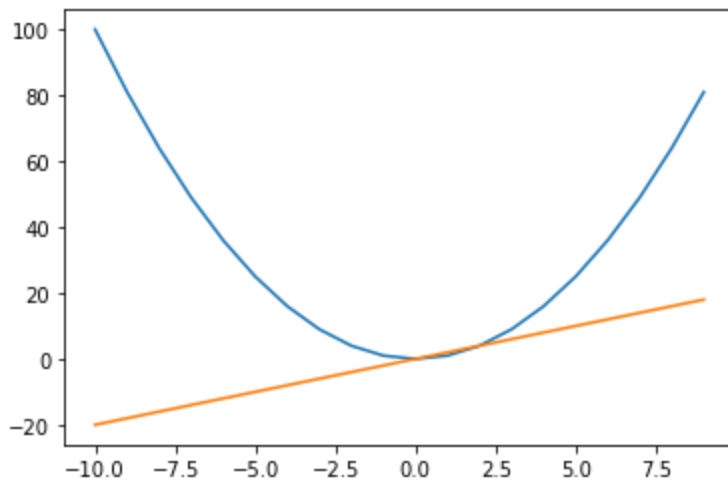


$f'(x) > 0 \Rightarrow$ minima
 $f'(x) < 0 \Rightarrow$ maxima

$$y = x^2$$

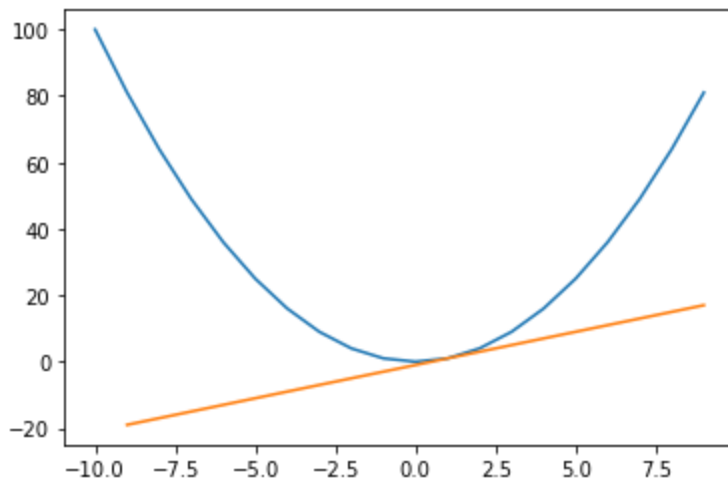
```
x = np.arange(-10,10)
y = x**2
plt.plot(x,y)
dy_dx = 2*x
plt.plot(x,dy_dx)
```

[<matplotlib.lines.Line2D at 0x16f71782ca0>]



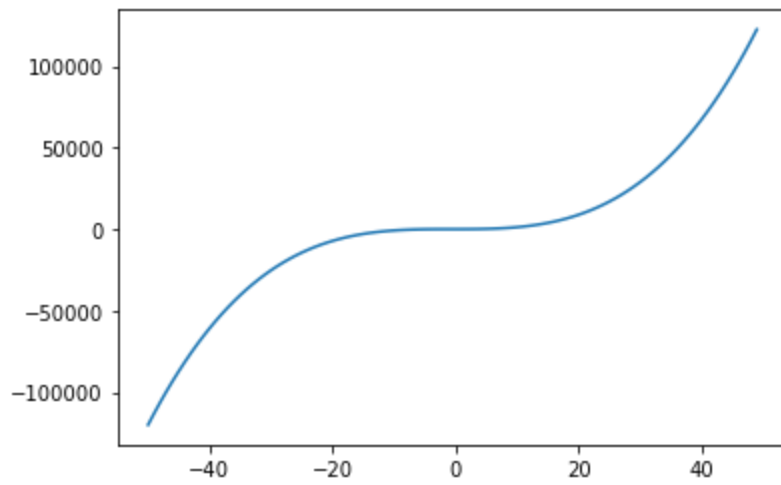
```
x = np.arange(-10,10)
y = x**2
plt.plot(x,y)
dy_dx = y[1:]-y[:-1]
plt.plot(x[1:],dy_dx)
```

[<matplotlib.lines.Line2D at 0x16f71621970>]



```
x = np.arange(-50,50)
y = x**3 + 2*(x**2) - 39
plt.plot(x,y)
```

[<matplotlib.lines.Line2D at 0x16f713980d0>]



```
pd.Series(y)
```

```
<IPython.core.display.Javascript object>
```

```
0    -120039
```

```
1    -112886
```

```
2    -106023
```

```
3     -99444
```

```
4     -93143
```

```
...
```

```
95     95136
```

```
96    101529
```

```
97    108202
```

```
98    115161
```

```
99    122412
```

```
Length: 100, dtype: int32
```

```
x = np.arange(-50,50)
```

```
y = x**2
```

```
plt.plot(x,y)
```

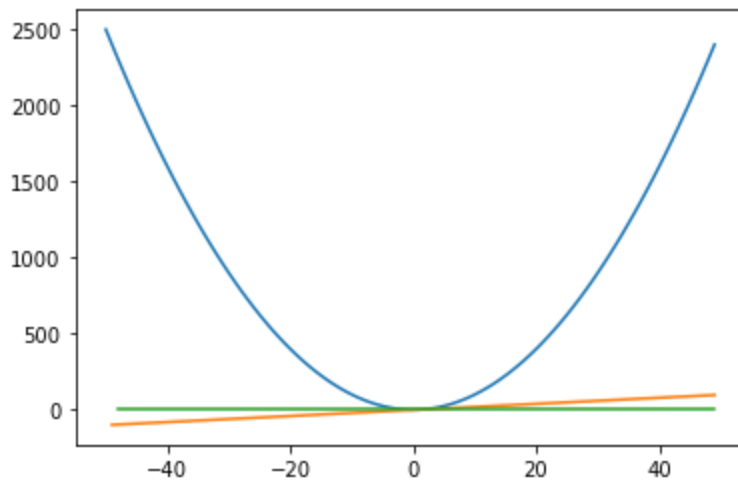
```
plt.plot(x,pd.Series(y).diff())
```

```
plt.plot(x,pd.Series(y).diff().diff())
```

```
<IPython.core.display.Javascript object>
```

```
<IPython.core.display.Javascript object>
```

```
[<matplotlib.lines.Line2D at 0x16f72607040>]
```



```
: # d/dx [ ((x-9)**2) + (5*(x-50)) + 785 ]
# = 2x-13
# 2x-13 = 0
# x = 13/2
```

```
((((x-9)**2) + (5*(x-50)) + 785).diff(x))
```

```
: 2x - 13
```

```
x = np.arange(-50,50)
```

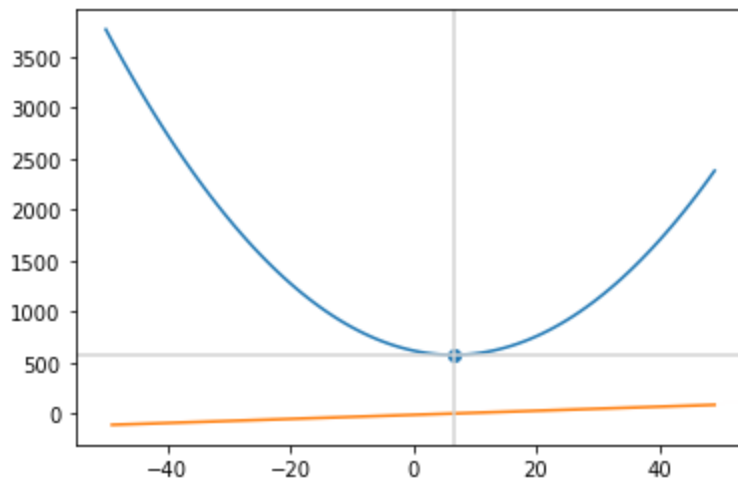
```
y = ((x-9)**2) + (5*(x-50)) + 785
```

```
plt.plot(x,y)
```

```
plt.plot(x,pd.Series(y).diff())
plt.axvline(x = 13/2, color = "lightgrey")
plt.axhline(y = 573.75, color = "lightgrey")
plt.scatter(x = 13/2, y = 573.75)
```

```
<IPython.core.display.Javascript object>
```

```
<matplotlib.collections.PathCollection at 0x16f73647400>
```



$$(((13/2)-9)**2) + (5*((13/2)-50)) + 785$$

$$573.75$$

$$4 - 250 + 785$$

$$539$$

```
x = np.arange(-50,50)
def f(x):
    return ((x-2)**2) + (5*(x-50)) + 785
y = f(x)

plt.plot(x,y)

'''
df(x)/dx = d/dx (x^2 - 2x + 4 + 5x - 250 + 785)
          = d/dx (x^2 + 3x + 539)
          = 2x + 3

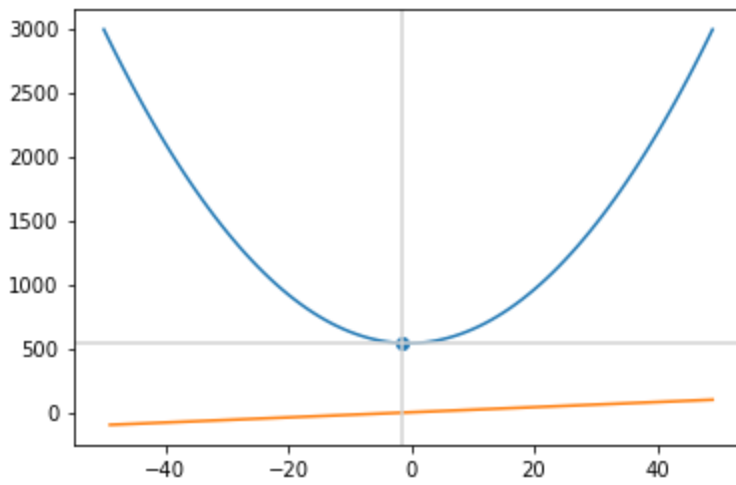
at slope is 0 , df(x)/dx = 0

so , 2x + 3 = 0
x = -3/2
'''
```

```
plt.plot(x,pd.Series(y).diff())
plt.axvline(x = -3/2, color = "lightgrey")
plt.axhline(y = f(-3/2), color = "lightgrey")
plt.scatter(x = -3/2, y = f(-3/2))
```

<IPython.core.display.Javascript object>

<matplotlib.collections.PathCollection at 0x16f736ffd30>



- 4 - 250 +785

531

```
x = np.arange(-50,50)
def f(x):
    return -(x-2)**2 + (5*(x-50)) + 785
y = f(x)
```

```
plt.plot(x,y)
```

'''

$$\begin{aligned} df(x)/dx &= d/dx (-x^2 + 2x - 4 + 5x - 250 + 785) \\ &= d/dx (-x^2 + 7x + 531) \\ &= -2x + 7 \end{aligned}$$

at slope is 0 , $df(x)/dx = 0$

so , $-2x + 7 = 0$
 $x = 7/2$

slope is negative : then its maxima

if slope is positive then its minima

we can also see, if double differentiation is +ve . then its minima

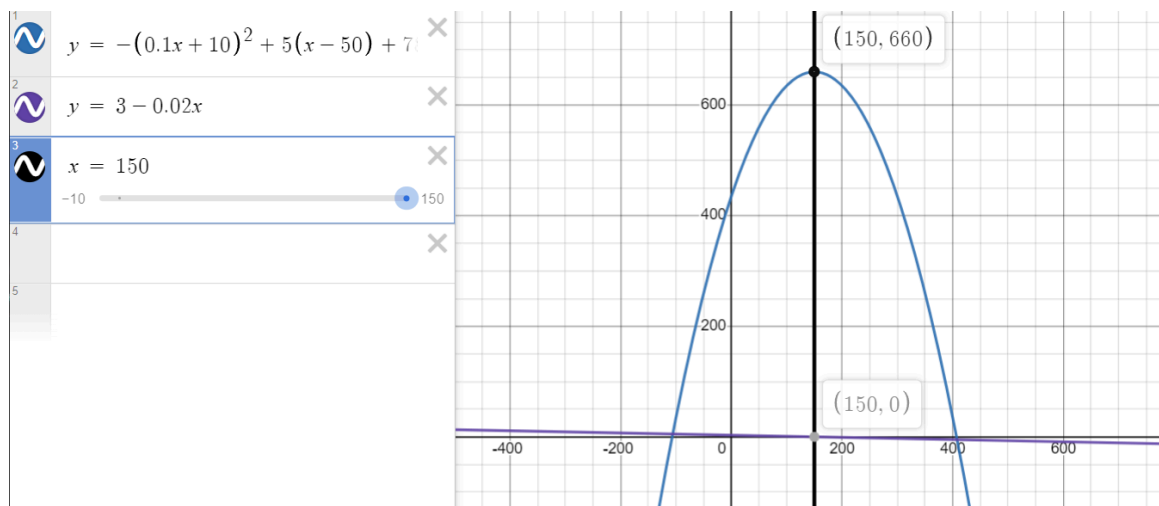
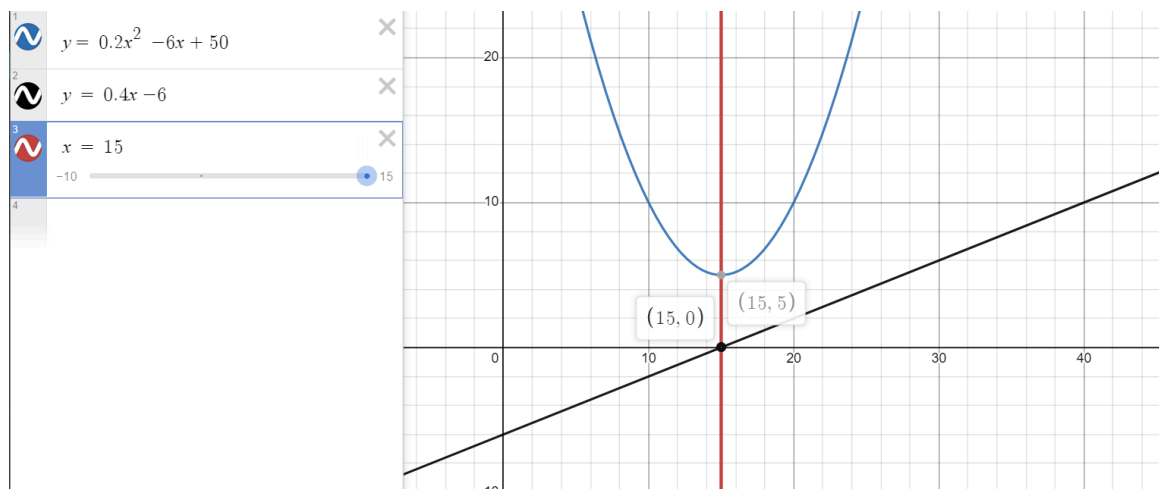
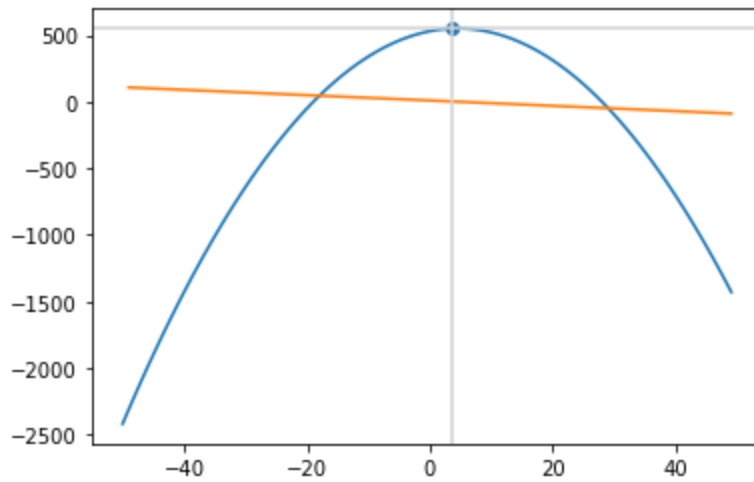
if double diff of a equation is -ve , then its maxima

'''

```
plt.plot(x,pd.Series(y).diff())
plt.axvline(x = 7/2, color = "lightgrey")
plt.axhline(y = f(7/2), color = "lightgrey")
plt.scatter(x = 7/2, y = f(7/2))
```

<IPython.core.display.Javascript object>

<matplotlib.collections.PathCollection at 0x16f7373aca0>



$$-(0.1x + 10)^2 + 5(x - 50) + 785$$

```
In [80]: ((-(0.1*x+10)**2) + (5*(x-50)) + 785).diff(x)
```

```
Out[80]: 3.0 - 0.02x
```

```
x = np.arange(-5000,5000)
```

```
def f(x):
    return ((-(0.1*x +10 )**2) + (5*(x-50)) + 785 )
```

```
y = f(x)
plt.plot(x,y)
plt.plot(x,pd.Series(y).diff())
```

```
'''
```

$dy/dx = 3 - 0.02x$

$3 - 0.02x = 0$

$x = 3/0.02$

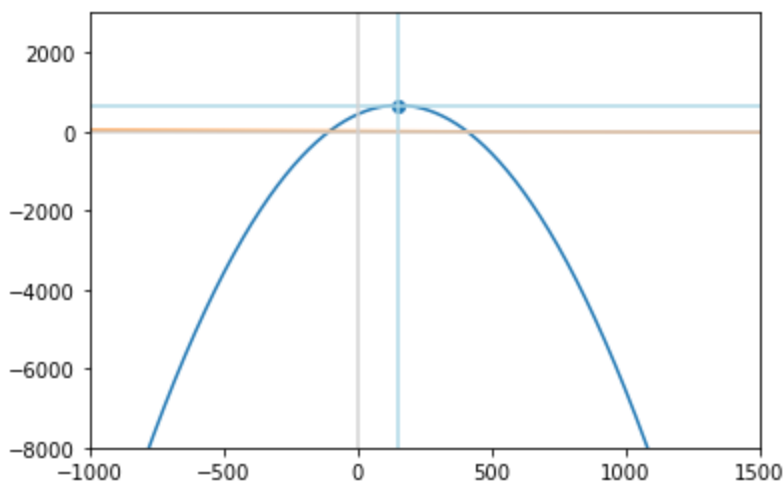
$x = 150$

```
'''
```

```
plt.axvline(x = 150, color = "lightblue")
plt.axhline(y = f(150), color = "lightblue")
plt.scatter(x = 150, y = f(150))
plt.axhline(y = 0,color = "lightgrey")
plt.axvline(x = 0, color = "lightgrey")
plt.xlim(-1000,1500)
plt.ylim(-8000,3000)
```

<IPython.core.display.Javascript object>

(-8000.0, 3000.0)



3/0.02

150.0

```
x = np.arange(-500,500)
```

```
def f(x):  
    return ((-(0.1*x +10 )**2) + (5*(x-50)) + 785 )
```

```
y = f(x)  
plt.plot(x,y)  
plt.plot(x,pd.Series(y).diff())
```

```
'''
```

$dy/dx = 3 - 0.02x$

$3 - 0.02x = 0$

$x = 3/0.02$

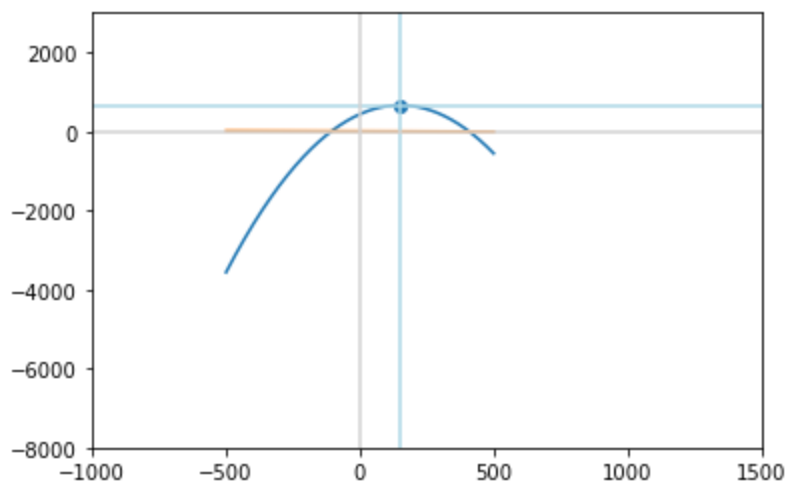
$x = 150$

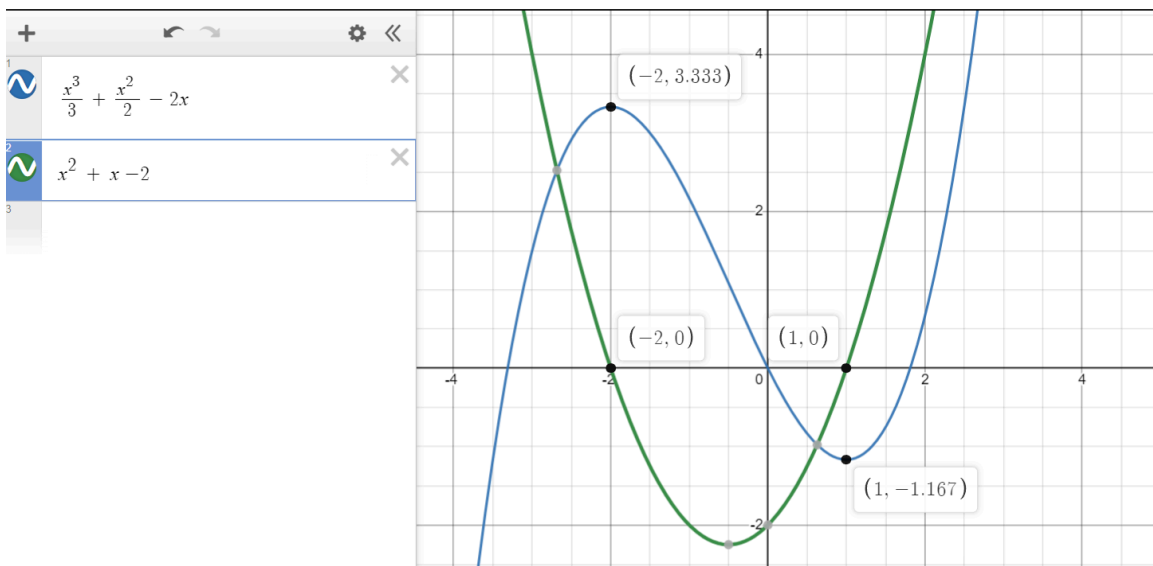
```
'''
```

```
plt.axvline(x = 150, color = "lightblue")  
plt.axhline(y = f(150), color = "lightblue")  
plt.scatter(x = 150, y = f(150))  
plt.axhline(y = 0,color = "lightgrey")  
plt.axvline(x = 0, color = "lightgrey")  
plt.xlim(-1000,1500)  
plt.ylim(-8000,3000)
```

<IPython.core.display.Javascript object>

(-8000.0, 3000.0)





```
x = np.arange(-10,10,0.1)
```

```
def f(x):
    return ((x**3) / 3 ) + ((x**2) / 2 ) - (2*x)
```

```
y = f(x)
plt.plot(x,y)
```

```
diff_y = (x**2) + x -2
plt.plot(x,diff_y)
```

```
doublediff_y = 2*x + 1
plt.plot(x,doublediff_y)
```

```
'''
```

```
dy/dx = x^2 + x -2
```

```
x^2 + x -2 = 0
```

```
x^2 + x = 2
```

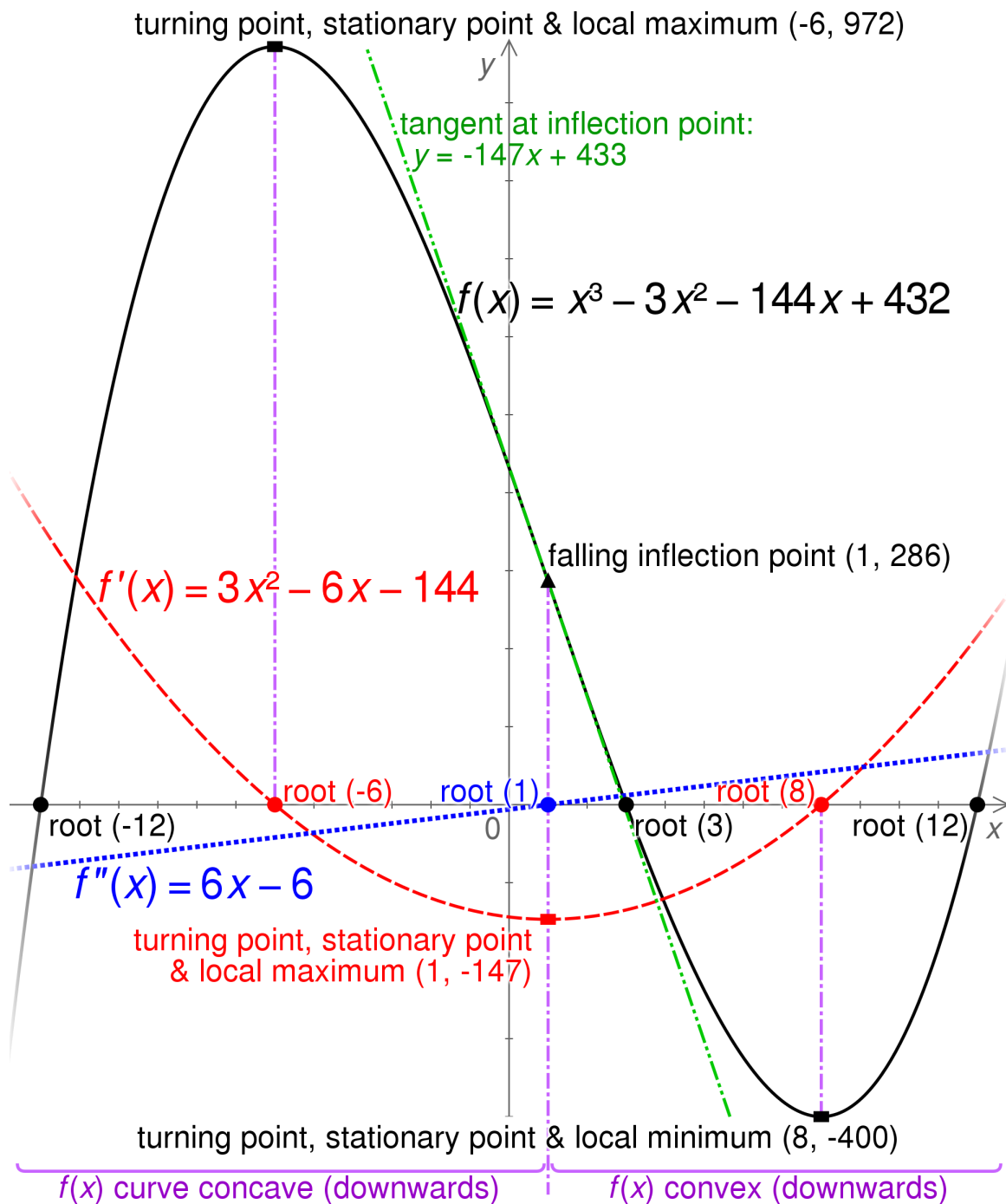
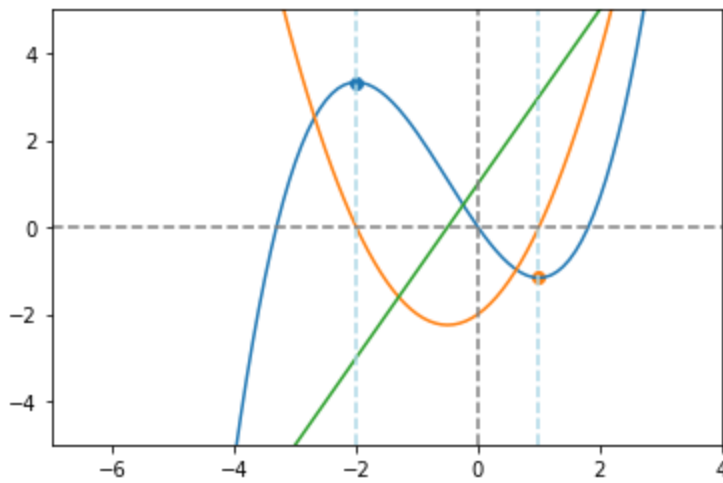
```
WE will hav two values of x , as x = -2 and +1
```

```
'''
```

```
plt.scatter(x = -2 , y = f(-2)) # maxima # inflation point
plt.scatter(x = 1 , y = f(1)) # minima
```

```
plt.axvline(x = -2,linestyle='dashed', color = "lightblue")
plt.axvline(x = 1,linestyle='dashed', color = "lightblue")
```

```
plt.axhline(y = 0,color = "grey",linestyle='dashed')
plt.axvline(x = 0, color = "grey",linestyle='dashed')
plt.xlim(-7,4)
plt.ylim(-5,5)
plt.show()
```



https://en.wikipedia.org/wiki/Inflection_point

```
from mpl_toolkits import mplot3d
```

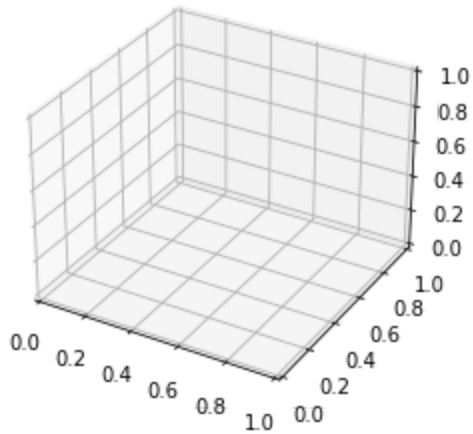
```
%matplotlib inline
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

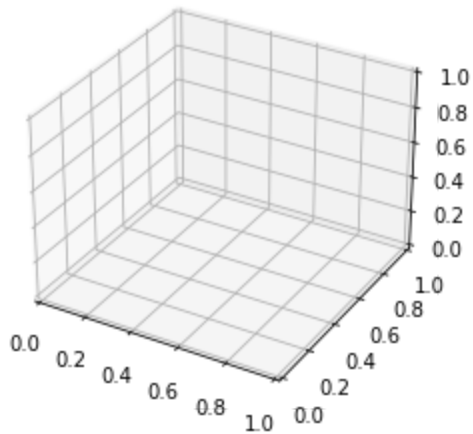
```
fig = plt.figure()
```

```
ax = plt.axes(projection='3d')
```



```
plt.axes(projection='3d')
```

```
<Axes3DSubplot:>
```



```
plt.figure(figsize=(10,10))
```

```
def f(x, y):
```

```
    return (np.sqrt(x ** 2 + y ** 2 + 50))
```

```
x = np.linspace(-5, 5, 30)
```

```
y = np.linspace(-5, 5, 30)
```

```
X, Y = np.meshgrid(x, y)
```

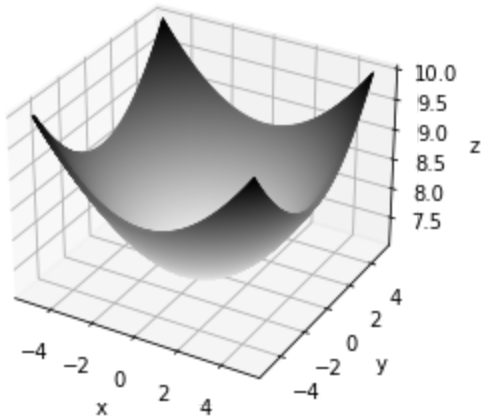
```
Z = f(X, Y)
```

```
fig = plt.figure()

ax = plt.axes(projection='3d')
ax.contour3D(X, Y, Z, 500, cmap='binary')

ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_zlabel('z')
plt.show()
```

<Figure size 720x720 with 0 Axes>



```
from sympy import Symbol
from sympy import init_printing
from sympy import integrate
from sympy import limit
from sympy import diff
```

```
init_printing(use_unicode=False, wrap_line=False)
x = Symbol('x')
y = Symbol('y')
```

```
function = (2*(x**2)*(y))+(3*(y**3)*(x**2))+(3*y)
```

$4xy + 6xy^3 + 3y$

```
function.diff(x)
```

$6xy^3 + 4xy$

```
function.diff(y)
```

$9x^2y^2 + 2x^2 + 3$

```
function = ((8*x**3) + (3*(x**2)*(y**2)) + (2*x) - (2*y) -1)
```

```
function.diff(x)
```

$$24x^2 + 6xy^2 + 2$$

$$f(x,y) = 8x^3 + 3x^2 y^2 + 2x - 2y - 1$$

$$24x^2 + 6xy^2 + 2$$

```
function = (x**2) - (3 *x*y)
```

```
function.diff(x), function.diff(y)
```

$$(2x - 3y, -3x)$$

if we have 3 features : x1 , x2 , x3

Howmany dimention will be there :

$$w_1x_1 + w_2x_2 + w_3x_3 + w_0 = 0$$

3 features + 1 intercept + 1 costfunction

total 5 dimention

For binary Classification

$$Data \rightarrow D = \{ (\vec{x}_i, y_i), \vec{x}_i \in R^d, y_i \in \{-1, 1\} \}$$

y_i is a target variable containing class or labels

$\vec{x}_i$ are the features that belongs to d dimension real numbers

$$f(\vec{x}_i) = \hat{y}_i \sim y_i$$

y_i : ground truth (target)

$\hat{y}_i$: predicted output

$f(\vec{x}_i)$ is the Machine learning Model which predicts the output

For classification: Mathematical problem equation

Optimization target / loss function

$$\text{Max over } \vec{w} \text{ and } w_0 ; \sum_{i=1}^n y_i \frac{w^T x_i + w_0}{||w||}$$

y_i is the predition that will be either - 1 or 1 in classification

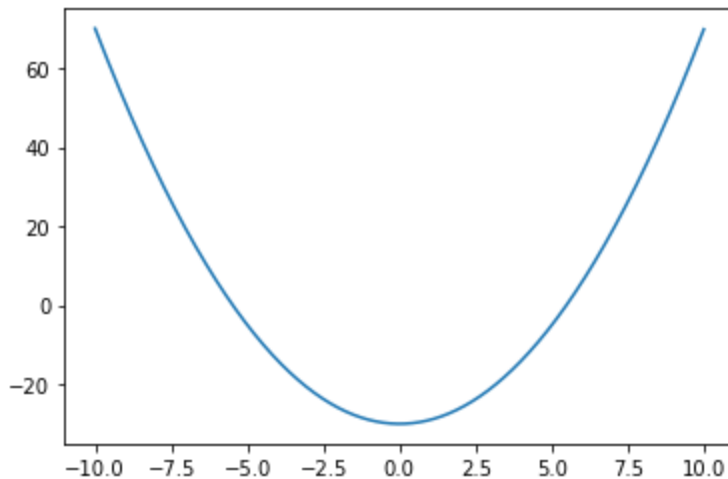
x_i is the input vector

Gradient descent :

we are trying to optimize above function , which is called Gain function .

Maximise distance of all points from the Decision boundry in the direction of their respective half space (class labels)

```
def f(x):  
    ## Try changing this  
    return x**2 - 30  
  
## Plot f(x)  
x = np.arange(-10, 10, 0.01)  
y = f(x)  
  
plt.figure()  
plt.plot(x, y)  
plt.show()
```



```
# This function takes derivative of f at point x  
def derivative(x, f):  
    delta = 0.0001  
    return (f(x + delta) - f(x)) / delta  
  
# This function does exactly one update to my parameter x  
def one_step_gd(x, f, lr): # lr -> Learning rate, same as \eta  
    grad = derivative(x, f)  
    new_x = x - lr * grad  
    new_fx = f(new_x)  
    return new_x, new_fx, grad  
  
new_x = np.random.randint(8, 15) # Initial x  
new_fx = f(new_x) # initial y  
num_steps = 0 # Same as t in the update equation  
  
# Let us save all our steps.  
xs = [new_x]
```

```
fxs = [new_fx]
grads = []
```

```
# First, plot the function
```

```
x = np.arange(-20, 20, 0.01)
```

```
# Now apply gradient descent
```

```
while True:
```

```
    new_x, new_fx, grad = one_step_gd(new_x, f, 0.05)
```

```
    xs.append(new_x)
```

```
    fxs.append(new_fx)
```

```
    grads.append(grad)
```

```
    num_steps += 1
```

```
    if len(fxs)>2 and abs(fxs[-1] - fxs[-2]) < 1e-3:
```

```
        break
```

```
    if num_steps > 100:
```

```
        print('Hard stop')
```

```
        break
```

```
print(f"Took {num_steps} iterations")
```

```
print(f"Final loss: {new_fx}")
```

```
print(f"Optimum value of parameter: {new_x}")
```

```
Took 47 iterations
```

```
Final loss: -29.9959579386359
```

```
Optimum value of parameter: 0.06357720789794996
```

```
import plotly.graph_objects as go
```

```
fig = go.Figure(
```

```
    data=[go.Scatter(x=x, y=f(x), mode="lines",
```

```
        line=dict(width=2,
```

```
        color="blue")) for i in range(len(xs))],
```

```
    layout=go.Layout(
```

```
        xaxis=dict(range=[-20, 20],
```

```
        autorange=False,
```

```
        zeroline=False),
```

```
        yaxis=dict(range=[-40, 140],
```

```
        autorange=False,
```

```
        zeroline=False),
```

```
        title_text = "Gradient Descent",
```

```
        hovermode="closest",
```

```
        updatemenus=[dict(type="buttons",
```

```
            buttons=[dict(label="Play",
```

```
                method="animate",
```

```
                args=[None])]]],
```

```
    frames=[go.Frame(
```

```
        data=[go.Scatter(
```

```
            x=[xs[k]],
```

```

        y=[fxs[k] + 5],
        mode="markers",
        marker=dict(color="red", size=20))])

    for k in range(len(xs))]
)

fig.show()

```

gradient descent visuals

https://github.com/lilipads/gradient_descent_viz

Loss function

best fit line / separation line :

$$y_i\text{-hat} = mx_i + b$$

$$\text{loss function} = \text{sum over } i=1 \text{ to } n [(y_i - y_i\text{-hat})^2]$$

$$L = \text{sum } 1 \text{ to } n [(y_i - mx_i - b)^2]$$

so , loss function is a function of parameters slope and intercept


```

from sklearn.datasets import make_regression

x,y = make_regression(n_samples = 100,
                      n_features = 1,
                      n_informative= 1,
                      n_targets=1,
                      noise=20,
                      )

# x

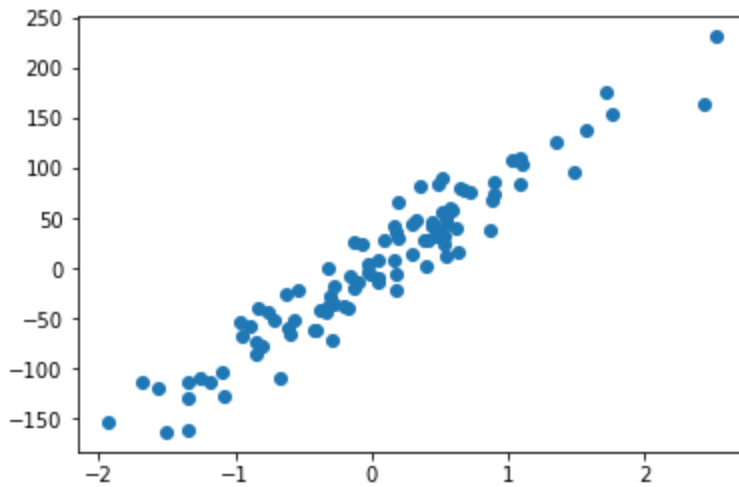
n = len(x)

# y

plt.scatter(x,y)

<matplotlib.collections.PathCollection at 0x16f7fe5bfd0>

```



```

reg = LinearRegression()

<IPython.core.display.Javascript object>

reg.fit(x,y)

LinearRegression()

reg.coef_

array([86.63215814])

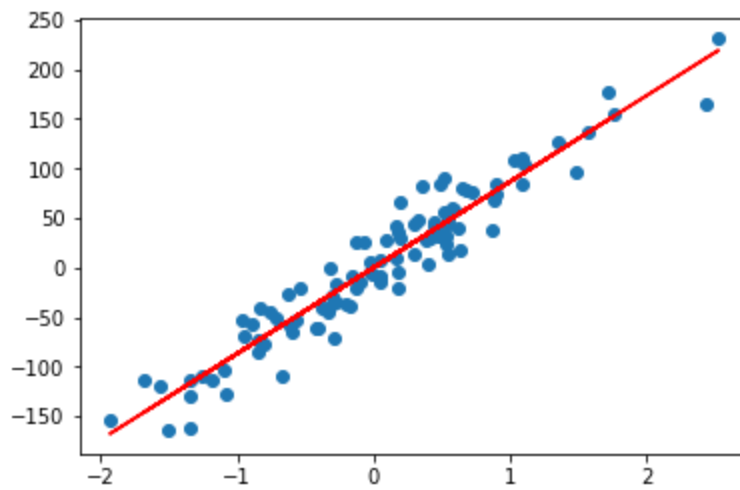
reg.intercept_

-0.201997965937344

plt.scatter(x,y)
plt.plot(x,reg.predict(x),color = "red")

```

```
[<matplotlib.lines.Line2D at 0x16f011b24c0>]
```



```
reg.coef_[0]
```

```
86.6321581449939
```

```
reg.intercept_
```

```
-0.201997965937344
```

```
prediction :
```

```
yi-hat = slope * xi + intercept
```

```
loss function :
```

```
L = sum from i = 1 to n [(yi- yi_hat )^2]
```

```
L = sum from i = 1 to n [(yi - mxi - b)^2]
```

```
L(slope, intercept) = { sum from i = 1 to n [(yi - mxi - b)^2] }
```

```
# Lets apply gradient assuming the slope is constant m = 41.43
```

```
# and start value of intercept b = 100
```

```
m = reg.coef_[0]
```

```
b = -200
```

```
y_pred = ((m * x) + b).reshape(len(x))
```

```
# we need find the value of intercept b ,
```

```
# such that loss function becomes minimum at that value of b
```

```
# y_pred
```

```
plt.scatter(x,y)
```

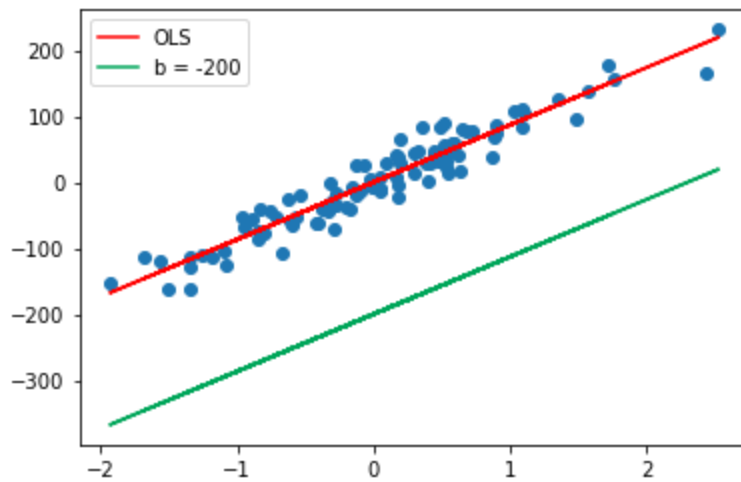
```
plt.plot(x,reg.predict(x),color = "red", label = "OLS")
```

```
plt.plot(x,y_pred, color = "#00a65a", label = "b = -200")
```

```
plt.legend()
```

```
plt.show()
```

```
# the intercept value we will find through intercept ,
# will be close to this red line
```



```
# y
```

```
# y_pred
```

```
loss = np.sum(((y-y_pred))**2)
loss
```

```
4041302.74270744
```

```
len(x),len(y)
```

```
(100, 100)
```

```
# bs = (np.arange(-5,5,0.01))
```

```
bs = (np.linspace(-5,5,n))
```

```
len(bs)
```

```
100
```

```
losses = []
```

```
for i in range(len(bs)):
```

```
    y_pred_ = ((m * x) + bs[i]).reshape(len(x))
```

```
    l = np.sum(((y-y_pred_))**2)
```

```
    losses.append(l)
```

```
# losses
```

```
len(losses)
```

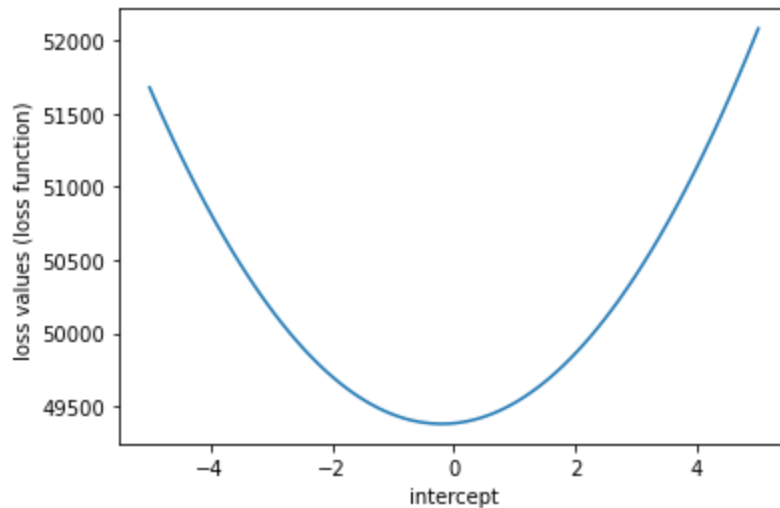
```
100
```

```
plt.plot(bs,losses) # loss function on y axis
```

```
plt.ylabel("loss values (loss function) ")
```

```
plt.xlabel("intercept")
```

```
Text(0.5, 0, 'intercept')
```



if we have negative slope , we need to increase the intercept here
if slope is positive , we need to decrease the intercept

```
b_new = b_old - slope
```

lets say we found a intercept b as -2

for next iteration

lets say we found slope as -3

new value of intercept will be

$b_{new} = -2 - (-3)$

so , new intercept will be 1 , that is in to +ve x direction

thats how iterarively we come more n more closer to minima

now , for small jumps

we transfor the bnew

```
bnew = b_old - eta * slope (eta is learning rate)
```

which minimise the jumps

ideally we keep learning rate at 0.01

:

now when to stop , since it is iterative algorithm

if $b_{new} - b_{old}$ tends to zero . (very minimum value)

algorithm is towards convergence

or we can limit iteration , lets say 1000 , and we estimate
algorithm will converge with in those iterations.
that is called epochs(iterations)

$b_{new} = b_{old} - \eta * \text{slope}$ here slope is the slope of curve
in lossfunction vs intercept graph
that we can find taking derivative
at each intercept value
(loss slope)

$$L = \sum ((y_i - \hat{y})^2)$$

$$dL/db = d/db (\sum (y_i - mx_i - b)^2)$$

slope:

$$\begin{aligned} dL/db &= 2(y_i - mx_i - b)(-1) \\ &= -2 (\sum (y_i - mx_i - b)) \end{aligned}$$

such way we find slopes at every b values

and find

$$b_{new} = b_{old} - LR * \text{slope} \quad (LR = \text{learning rate})$$

and

$LR * \text{slope}$ is called step size

$$b_{new} = (b_{old} - (LR * (dL/db)))$$

m

86.6321581449939

b

-200

```
In [628]: import sympy as sy

init_printing(use_unicode=False, wrap_line=False)
x = Symbol('x')
```

```
In [633]: b = Symbol('b')
y = Symbol("y")
m = Symbol("m")
```

```
In [634]: ((y-(m*x)-(b))**2).diff(b)
```

```
Out[634]: 2b + 2mx - 2y
```

```
In [ ]: -2(-b-mx+y)
        -2(y-mx-b)
```

```
loss_slope = -2 * (np.sum(y - m*x.ravel() - b))
```

```
loss_slope
```

```
-39959.6004068125
```

```
# Lets take Learning rate as 0.1
```

```
LR = 0.001
```

```
step_size = loss_slope * LR
```

```
step_size
```

```
-39.9596004068125
```

```
b = b-step_size
```

```
b # b_new
```

```
-160.040399593187
```

```
y_pred2 = ((m * x) + b).reshape(len(x))
```

```
plt.scatter(x,y)
```

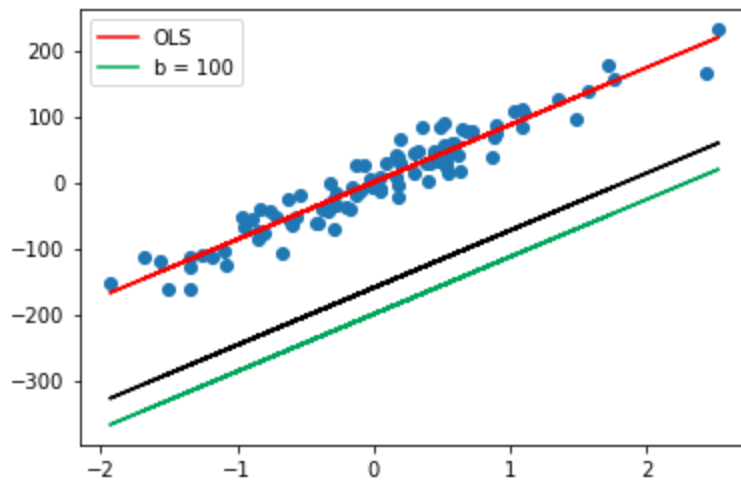
```
plt.plot(x,reg.predict(x), color = "red",label = "OLS")
```

```
plt.plot(x,y_pred, color = "#00a65a", label = "b = 100")
```

```
plt.plot(x,y_pred2, color = "black")
```

```
plt.legend()
```

```
plt.show()
```



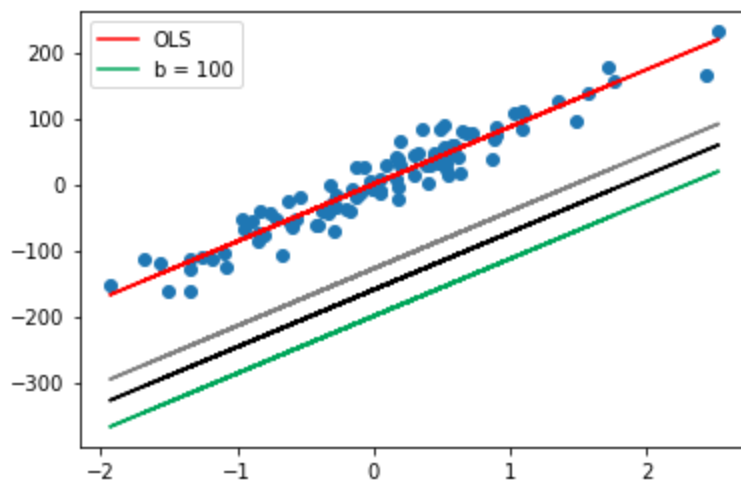
*# here we had started at $b = 100$,
and moving towards correct line which is red*

#iteration 2 :

```
loss_slope = -2 * (np.sum(y - m*x.ravel() - b))
LR = 0.001
step_size = loss_slope * LR
step_size
b = b-step_size
y_pred3 = ((m * x) + b).reshape(len(x))

plt.scatter(x,y)
plt.plot(x,reg.predict(x), color = "red",label = "OLS")
plt.plot(x,y_pred, color = "#00a65a", label = "b = 100")
plt.plot(x,y_pred2, color = "black")
plt.plot(x,y_pred3, color = "grey")

plt.legend()
plt.show()
```



iteration 3

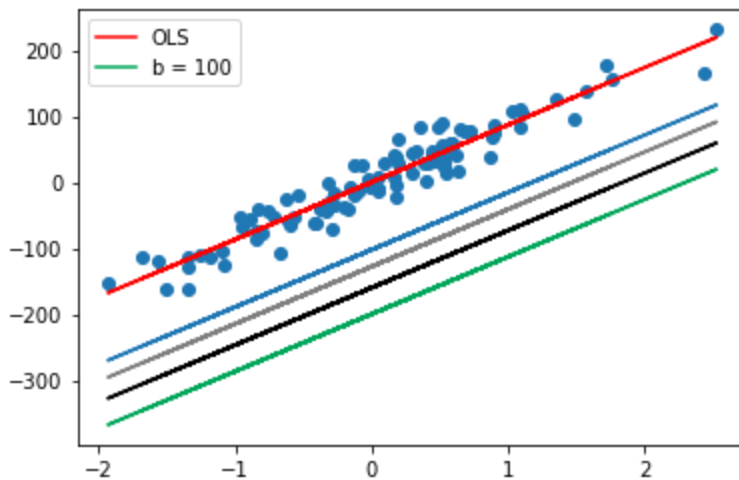
```

loss_slope = -2 * (np.sum(y - m*x.ravel() - b))
LR = 0.001
step_size = loss_slope * LR
step_size
b = b-step_size
y_pred4 = ((m * x) + b).reshape(len(x))

plt.scatter(x,y)
plt.plot(x,reg.predict(x), color = "red",label = "OLS")
plt.plot(x,y_pred, color = "#00a65a", label = "b = 100")
plt.plot(x,y_pred2, color = "black")
plt.plot(x,y_pred3, color = "grey")
plt.plot(x,y_pred4)

plt.legend()
plt.show()

```



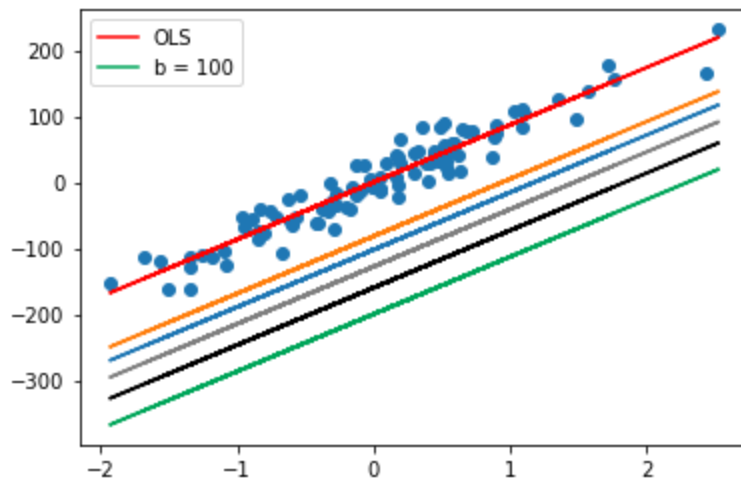
```

loss_slope = -2 * (np.sum(y - m*x.ravel() - b))
LR = 0.001
step_size = loss_slope * LR
step_size
b = b-step_size
y_pred5 = ((m * x) + b).reshape(len(x))

plt.scatter(x,y)
plt.plot(x,reg.predict(x), color = "red",label = "OLS")
plt.plot(x,y_pred, color = "#00a65a", label = "b = 100")
plt.plot(x,y_pred2, color = "black")
plt.plot(x,y_pred3, color = "grey")
plt.plot(x,y_pred4)
plt.plot(x,y_pred5)

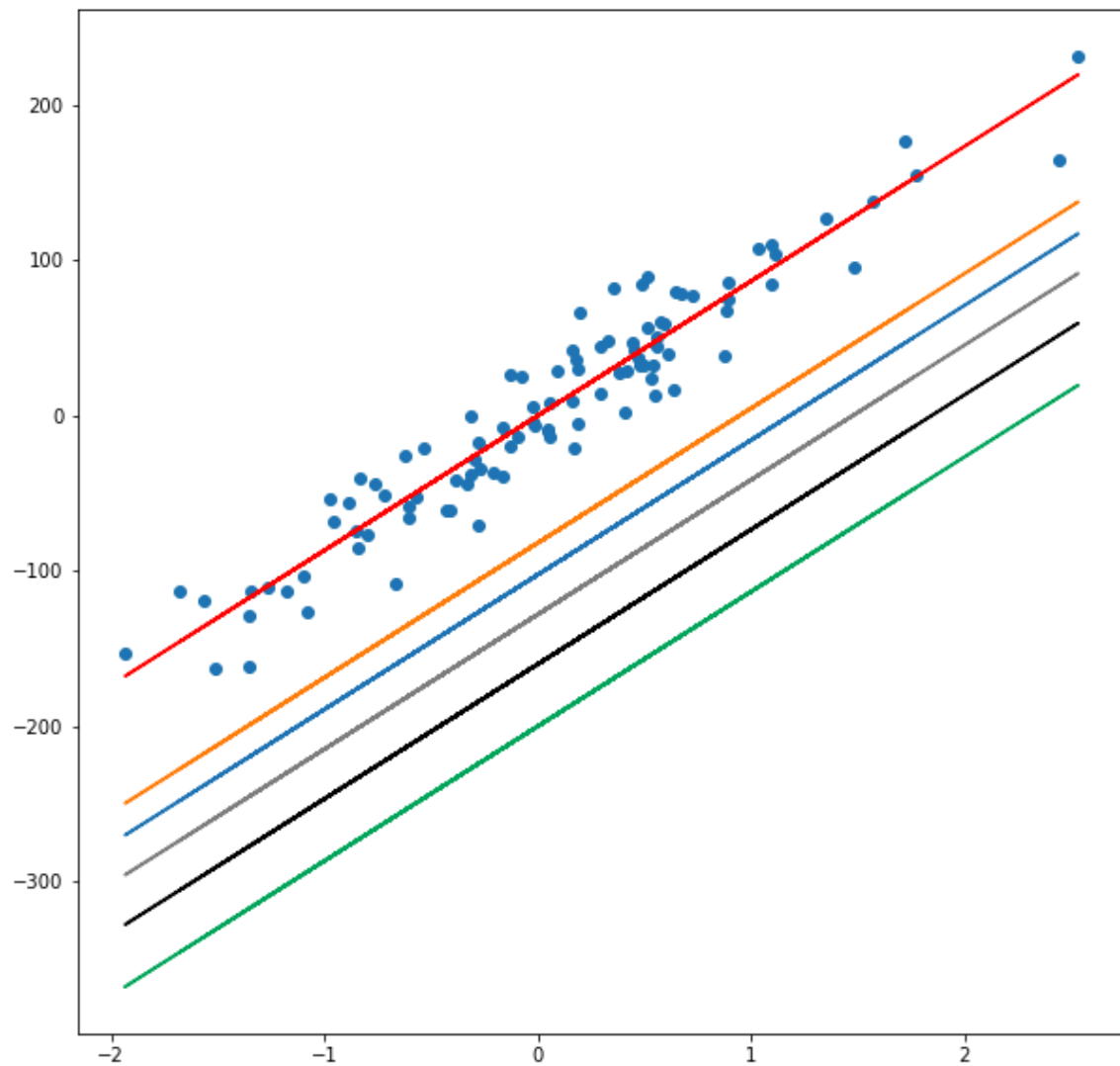
plt.legend()
plt.show()

```

```
plt.figure(figsize=(10,10))
plt.scatter(x,y)
plt.plot(x,reg.predict(x), color = "red",label = "OLS")
plt.plot(x,y_pred, color = "#00a65a", label = "b = 100")
plt.plot(x,y_pred2, color = "black")
plt.plot(x,y_pred3, color = "grey")
plt.plot(x,y_pred4)
plt.plot(x,y_pred5)
```

[<matplotlib.lines.Line2D at 0x16f7fffcbe0>]



b

-82.0392595990894

reg.intercept_

-0.201997965937344

m

86.6321581449939

b = 500

lr = 0.001

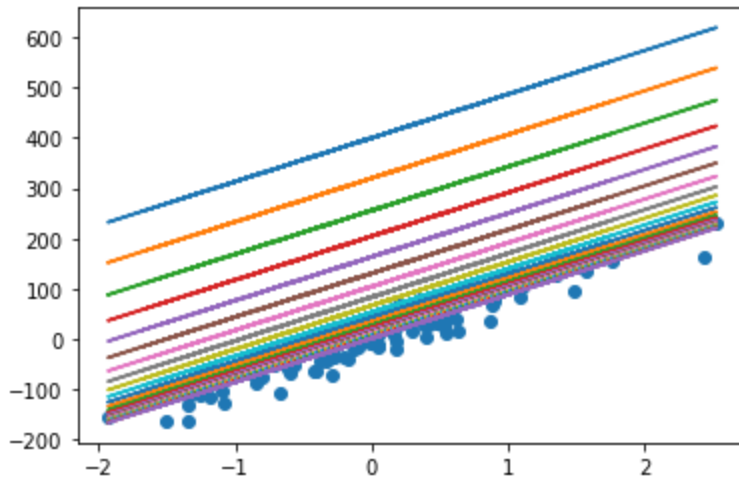
epochs = 25

```
for i in range(epochs):
    loss_slope = -2 * np.sum(y-m*x.ravel()-b)
    b = b-(lr*loss_slope)

    y_predd = m*x + b
```

```
plt.plot(x,y_predd)
plt.scatter(x,y)

<matplotlib.collections.PathCollection at 0x16f7fe66910>
```

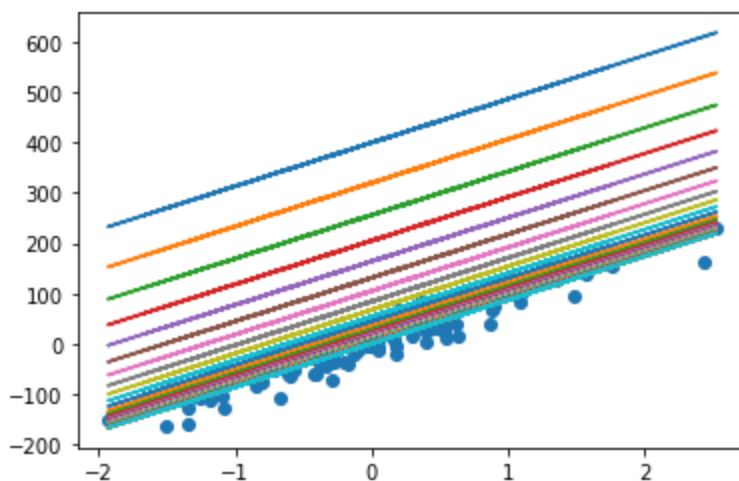


```
b = 500
lr = 0.001
epochs = 50

for i in range(epochs):
    loss_slope = -2 * np.sum(y-m*x.ravel()-b)
    b = b-(lr*loss_slope)

    y_predd = m*x + b
    plt.plot(x,y_predd)
plt.scatter(x,y)

<matplotlib.collections.PathCollection at 0x16f012ff670>
```



```
b
-0.194858844462506
reg.intercept_
```

-0.201997965937344

finding Gradient with slope and intercept together together :

```
In [628]: import sympy as sy

init_printing(use_unicode=False, wrap_line=False)
x = Symbol('x')
```

```
In [633]: b = Symbol('b')
y = Symbol("y")
m = Symbol("m")
```

```
In [634]: ((y-(m*x)-(b))**2).diff(b)
```

```
Out[634]: 2b + 2mx - 2y
```

```
In [ ]: -2(-b-mx+y)
        -2(y-mx-b)
```

x,m,b

(x, m, b)

$((y-(m*x)-(b))**2).diff(m)$ *# slope wrt m @ m value*

$-2x(-b - mx + y)$

$((y-(m*x)-(b))**2).diff(b)$ *# slope wrt b @ b value*

$2b + 2mx - 2y$

$dL/dm = -2x(y-b-mx)$

$dL/db = -2(y-mx-b)$